URL-Scoped Web Assessment Report

Report ID: 492dc3d12a404e48b91a6cc7490c4a89

Generated: 2026-10-12 05:52:09 UTC

Exact target: https://pentest-ground.com:4280/

Executive summary

Coverage: 5 recorded tool steps; 5 completed, 0 failed, 0 unknown.

Outcome: No confirmed vulnerabilities were established by the limited checks performed. This result does not determine whether other paths are vulnerable.

Scope: Checks used the supplied URL only and did not crawl linked pages or test authenticated routes.

Transport observation: TLS certificate validation succeeded for the tested HTTPS response.

Security-header configuration observation: the response did not include Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy. Missing headers can reduce browser-side hardening in relevant contexts; impact depends on application behavior and other controls. This observation alone does not establish a vulnerability.

Technology fingerprint inventory: PHP/8.5.11, nginx/1.31.6. This is technology inventory only; product or banner identification does not establish a vulnerability.

XSS probe: the harmless marker was not observed in the tested response; this does not rule out other XSS paths.

SQL injection probe: no response-length signal was flagged by this limited check.

No CSRF-specific check was performed.

Recorded tool outcomes

Statuses and evidence below come from the recorded tool steps. UNKNOWN means no success or failure status was recorded.

1. open_url - COMPLETED

Recorded arguments:

```
{
  "url": "https://pentest-ground.com:4280/"
}
```

Recorded tool output (evidence):

```
success: True
title: Welcome :: Damn Vulnerable Web Application (DVWA)
content:
  Home
  Instructions
  Setup / Reset DB
  Brute Force
  Command Injection
  CSRF
  File Inclusion
  File Upload
  Insecure CAPTCHA
  SQL Injection
  SQL Injection (Blind)
  Weak Session IDs
  XSS (DOM)
  XSS (Reflected)
  XSS (Stored)
```

```
  CSP Bypass
  JavaScript
  Open HTTP Redirect
  DVWA Security
  PHP Info
  About
  Logout

  Welcome to Damn Vulnerable Web Application!
  Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable.
  Its main goal is to be an aid for security professionals to test their skills and tools in a
  legal environment, help web developers better understand the processes of securing web
  applications and to aid both students & teachers to learn about web application security in a
  controlled class room environment.
  The aim of DVWA is to practice some of the most common web vulnerabilities, with various
  levels of difficultly, with a simple straightforward interface.

  General Instructions
  It is up to the user how they approach DVWA. Either by working through every module at a fixed
  level, or selecting any module and working up to reach the highest level they can before
  moving onto the next one. There is not a fixed object to complete a module; however users
  should feel that they have successfully exploited the system as best as they possible could by
  using that particular vulnerability.
  Please note, there are both documented and undocumented vulnerability with this software. This
  is intentional. You are encouraged to try and discover as many issues as possible.
  There is a help button at the bottom of each page, which allows you to view hints & tips for
  that vulnerability. There are also additional links for further background reading, which
  relates to that security issue.

  WARNING!
  Damn Vulnerable Web Application is damn vulnerable! Do not upload it to your hosting
  provider's public html folder or any Internet facing servers, as they will be compromised. It
  is recommend using a virtual machine (such as VirtualBox or VMware), which is set to NAT
  networking mode. Inside a guest machine, you can download and install XAMPP for the web server
  and database.

  Disclaimer
  We do not take responsibility for the way in which any one uses this application (DVWA). We
  have made the purposes of the application clear and it should not be used maliciously. We have
  given warnings and taken measures to prevent users from installing DVWA on to live web
  servers. If your web server is compromised via an installation of DVWA it is not our
  responsibility it is the responsibility of the person/s who uploaded and installed it.

  More Training Resources
  DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications.
  However there are plenty of other issues with web applications. Should you wish to explore any
  additional attack vectors, or want more difficult challenges, you may wish to look into the
  following other projects:

  Mutillidae
  OWASP Vulnerable Web Applications Directory

  Username: UnknownSecurity Level: lowLocale: enSQLi DB: mysql

  Damn Vulnerable Web Application (DVWA)
truncated: False
```

2. web_security_headers - COMPLETED

Recorded arguments:

```
{
  "url": "https://pentest-ground.com:4280/"
```

```
}
```

Recorded tool output (evidence):

```
url: https://pentest-ground.com:4280/
status_code: 200
final_url: https://pentest-ground.com:4280/
server: nginx/1.31.6
tls_verification: verified
security_headers:
  Strict-Transport-Security: MISSING
  Content-Security-Policy: MISSING
  X-Frame-Options: MISSING
  X-Content-Type-Options: MISSING
  Referrer-Policy: MISSING
  Permissions-Policy: MISSING
missing_security_headers:
  - Strict-Transport-Security
  - Content-Security-Policy
  - X-Frame-Options
  - X-Content-Type-Options
  - Referrer-Policy
  - Permissions-Policy
score: 0
```

3. web_tech_fingerprint - COMPLETED

Recorded arguments:

```
{
  "url": "https://pentest-ground.com:4280/"
}
```

Recorded tool output (evidence):

```
url: https://pentest-ground.com:4280/
technologies:
  -
    name: PHP/8.5.11
    category: backend
  -
    name: nginx/1.31.6
    category: server
detected: True
tls_verification: verified
```

4. xss_scanner - COMPLETED

Recorded arguments:

```
{
  "url": "https://pentest-ground.com:4280/",
  "method": "GET",
  "param": "q",
  "payload": "SVS_XSS_REFLECTION_PROBE"
}
```

Recorded tool output (evidence):

```
success: True
output:
  url: https://pentest-ground.com:4280/
  reflected: False
  status: 200
  note: Reflection is not proof of executable XSS.
```

5. sqli_scanner - COMPLETED

Recorded arguments:

```
{
```

```
  "url": "https://pentest-ground.com:4280/",
  "param": "id"
}
```

Recorded tool output (evidence):

```
success: True
output:
  url: https://pentest-ground.com:4280/
  param: id
  response_length_delta: 0
  possible_sqli: False
```

Evidence interpretation

- Transport observation: TLS certificate validation succeeded for the tested HTTPS response.

- Security-header configuration observation: the response did not include Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy. Missing headers can reduce browser-side hardening in relevant contexts; impact depends on application behavior and other controls. This observation alone does not establish a vulnerability.

- Technology fingerprint inventory: PHP/8.5.11, nginx/1.31.6. This is technology inventory only; product or banner identification does not establish a vulnerability.

- XSS probe: the harmless marker was not observed in the tested response; this does not rule out other XSS paths.

- SQL injection probe: no response-length signal was flagged by this limited check.

Recommended follow-up

- Review the missing response headers against the application's behavior and deployment; test CSP and frame protections before enforcing them.

- For broader coverage, assess discovered paths and authenticated workflows with authorized test credentials. This run did not crawl linked pages or log in.

- Validate input handling on application-specific parameters and forms. Test CSRF separately in a controlled session; this run made no state-changing requests.

Scope and limitations

No findings are inferred or added by this report writer. No CSRF-specific check was performed. Tool output is recorded evidence and may require analyst validation. A missing, failed, or inconclusive check does not establish that a vulnerability exists or that the target is free of vulnerabilities.